

Architecture Document: AI Intelligence Graph Pipeline

1. Overview

This document describes the architecture for a scalable data ingestion pipeline that collects, extracts, and structures information about startups, products, research papers, jobs, and news for the AI/VC ecosystem.

2. Scale Strategy: Collecting 500,000+ Records

Horizontal scaling via worker pools, not code changes.

The scraper is built on `asyncio` + `aiohttp`, which means a single process can already run hundreds of concurrent requests. To scale from ~1,000 records (this trial) to 500,000+, the same code runs across multiple worker processes/ containers, each assigned a partition of the work:

- **Partitioning by source:** each data vertical (startups, products, papers, jobs, news) runs as an independent worker pool, since they hit different sites and have different rate limits.
- **Partitioning by pagination range:** for a paginated source like `paperswithcode.co/papers/recent`, worker N is assigned pages `[N*100, (N+1)*100)` so workers never overlap and no central coordinator is needed for page assignment.
- **Queue-based work distribution:** for less predictable sources (crawling outward from a startup directory, following links), discovered URLs are pushed to a shared queue (Redis or SQS) and any idle worker pulls the next job. This lets us add more worker containers under load without code changes -- purely an infrastructure/deployment decision (e.g., scaling a Kubernetes deployment's replica count).
- **Concurrency limits per worker:** each worker uses an `asyncio.Semaphore` (demonstrated in `scraper.py`'s `scrape_papers_bulk`) to cap simultaneous requests to any one host, so scaling workers doesn't turn into an accidental DDoS on a source site.

3. Handling 413s and 429s at Scale

429 (Rate Limiting): Every external call (scrapping, GitHub API, LLM API) is wrapped in a retry function using exponential backoff with jitter, implemented in `llm_orchestrator.py`'s `call_with_retry`. On a 429, the worker waits $2^{\text{attempt}} + \text{random_jitter}$ seconds before retrying, up to a max retry count, then falls through to the next provider (for LLM calls) or logs the failure for a later re-run.

For GitHub's API specifically, unauthenticated requests are capped at 60/hour/IP -- at scale this requires either (a) authenticated calls with a GitHub personal access token (5,000/hour), or (b) rotating a pool of tokens across workers, each token used by a different worker.

413 (Payload Too Large): Before sending any text to an LLM, `llm_orchestrator.py`'s `chunk_text` function splits content into chunks under a safe character limit (~12,000 chars, roughly 3,000 tokens), breaking on paragraph boundaries where possible to preserve semantic coherence. Each chunk is processed and extracted independently, then results are merged. This means payload size never depends on the size of the source document -- a 50-page report and a tweet are handled by the same code path.

4. Freshness Tracking: Avoiding Duplicate Processing

Across distributed crawler nodes, duplicate work is avoided via:

- **Content-addressed deduplication:** each scraped item is hashed (URL + normalized title, or a content hash of the extracted text) and checked against a shared "seen" store (Redis set, or a `collected_urls` table in the primary database) before processing. If the hash exists, the worker skips it.

- **Watermarking by timestamp:** for the 24-hour freshness requirement (news/jobs), each worker records the last successful run time per source. On the next run, only items published after that watermark are fetched (using the source's own sort-by-date ordering, as seen in paperswithcode.co/papers/recent, which is already sorted newest-first -- a worker can stop paginating as soon as it sees an already-seen item).
- **Idempotent writes:** the primary database write uses an upsert keyed on the source URL, so even if two workers somehow scrape the same item, the second write overwrites rather than duplicates.

5. Storage Strategy

Primary database: PostgreSQL Structured, relational data (startups, products, papers, jobs) fits a relational model well, and Postgres gives us: - Strong consistency for upserts/dedup checks - JSONB columns for flexible fields (e.g., `all_linked_repos` in the paper schema) without a rigid schema migration for every new field - Mature tooling for the analytics team downstream

Vector storage: a dedicated vector DB (e.g., Pinecone, Weaviate, or pgvector as a Postgres extension) For semantic search and similarity-based entity resolution -- e.g., finding that "OpenAI's ChatGPT" and "GPT product line" refer to related entities even without exact string matches, we generate embeddings from entity descriptions and store them for nearest-neighbor lookup. Starting with pgvector keeps infrastructure simpler for the trial; a dedicated vector DB is a natural next step once the entity graph grows past what a single Postgres instance comfortably indexes.

Graph relationships: Neo4j (or a graph layer over Postgres) The core value described in the brief -- "Intelligence Graph" -- implies relationships matter as much as entities: which founders came from which companies, which products belong to which startups, which papers cite which repos. A graph database makes multi-hop queries (e.g., "show me all products from startups founded by ex-OpenAI employees") tractable in a way that repeated SQL joins are not. For this trial's scope, these relationships are captured as foreign keys in Postgres; a dedicated graph layer is the recommended evolution once relationship-traversal queries become common.

6. Anti-Bot Strategy

For this trial, sources were selected that expose either clean structured data (JSON-LD embedded in paperswithcode.co pages) or public APIs (GitHub's REST API for star counts), avoiding the need for browser automation entirely for Phase I.

For sources that do employ Cloudflare/Datadome protection, the strategy is: - **Playwright in async mode** (`playwright.async_api`), which renders full JS and can pass basic bot-detection heuristics that a raw `aioshttp` request cannot (executing JS challenges, realistic navigation timing). - **Residential proxy rotation** for high-value sources that aggressively block datacenter IPs -- the crawler's HTTP client is configured to route through a rotating proxy pool, changing the apparent origin IP per request or per session. - **Realistic request pacing:** randomized delays between requests (rather than fixed intervals) and respecting `robots.txt` crawl-delay directives where present, reducing the fingerprint that flags automated traffic. - **Session persistence:** reusing cookies/session tokens across requests to the same domain, mimicking a real browsing session rather than stateless one-off requests.

7. Lessons From This Trial

While building this pipeline, the original expected source (paperswithcode.com) turned out to be defunct -- it now redirects to Hugging Face's Trending Papers page, following Meta's shutdown of the original service in mid-2025. The community-relaunched successor, paperswithcode.co (built by Hugging Face's open-source team), restored much of the original functionality including code-repository linking. This is a concrete example of the kind of "navigate ambiguity" the brief calls out: the task doc's own example URL (paperswithcode.co/paper/98456) pointed to the correct, working domain, which required verifying independently rather than assuming the well-known .com domain still worked as expected.

